

RetireSafe

An evidence-driven workflow for safely retiring cloud resources

Solution dossier & proposed architecture — grounded in three test beds run on **real** public data (Cisco Umbrella top-1M, the NASA-HTTP 1995 traffic archive, and the *can-i-take-over-xyz* fingerprint catalogue). No figures in this document are fabricated; every number traces to a script and an output file in `/testbeds`.

~0.03% reclaimable references found across 16,000 real live hostnames (TB1)	42–127 d silence needed to be 99% sure a resource is truly dead (TB2)	108 / 108 'idle' resources that still had hidden external dependents (TB3)
--	--	---

THE ONE-LINE PROBLEM

*When a cloud resource is deleted, the **trust placed in its address survives** — DNS records, app configs, SPF rules, and automated clients keep pointing at a name that an attacker can now reclaim. RetireSafe decides, with evidence and stated uncertainty, whether a resource is safe to release.*

What this dossier contains

- **§1 Problem model** — the five conditions for takeover, as a checkable state machine.
- **§2 Evidence & scale** — what the public record establishes, stated conservatively.
- **§3 Three test beds on real data** — the heart of the submission: measured base rates, a traffic-survival model, and the coordination gap quantified.
- **§4 Proposed architecture** — scanners, the evidence graph, the decision engine, outputs.
- **§5 The mathematics** — Poisson survival, Wilson bounds, risk scoring, calibration.
- **§6 Datasets & tools, §7 roadmap, §8 honesty boundaries, §9 sources.**

The problem, as a checkable model

A broken reference is not automatically a vulnerability. Takeover requires a **conjunction** of five conditions. RetireSafe's job is to evaluate each one with the evidence available, and to report the ones it *cannot* evaluate rather than guessing. We model a retirement candidate as passing only if every condition is provably false.

#	Condition for takeover	What RetireSafe checks	Evidence source
1	Resource released	Would a different account be able to obtain the name?	Provider API / namespace rules
2	Name reassignable	Reserved names, domain verification, account-regional S3	Fingerprint catalogue + provider docs
3	Reference survives	DNS/CNAME, app config, SPF, code, IaC still point there	DNS, repo scan, config export
4	Consumer remains	Someone/something still relies on the reference	Access-log traffic analysis
5	Controls permit impact	No owner/integrity check rejects the replacement	bucket-owner-condition, SRI, sig

Design consequence: the output is never a binary 'safe'. It is a **per-condition verdict** — **cleared**, **blocking**, or **unknown** — so an engineer sees exactly why a deletion is held and what evidence would release it.

What the public record establishes

Drawn from the research brief's own sources and re-stated at the strength the evidence actually supports:

- **Malicious exploitation is real.** Infoblox documented the **CDC** Azure-endpoint takeover (Feb–Mar 2025) and the broader **Hazy Hawk** campaign (active since Dec 2023) hijacking abandoned DNS references of CDC, a French government Olympics site, and an oercommons S3 bucket. These are subdomain hijacks — not proof of internal-network breach.
- **It happens at scale.** USENIX NSDI 2024 (Friess et al.) identified **20,904** hijacked cloud resources, ~■ persisting >65 days; 75% used for blackhat SEO. That is a count of instances, not of breached organisations.
- **Dependencies outlive projects by years.** watchTowr registered ~150 abandoned S3 buckets and received **8M+ HTTP requests** in two months; a mozilla-games bucket was still receiving traffic years after its doc reference was removed in 2015.
- **Email trust survives too.** Guardio's SubdoMailing found 8,000+ domains with dangling SPF/CNAME authorisation (MSN, Swatch) abused to pass SPF on spoofed mail.
- **Observed traffic ≠ compromise.** watchTowr's .mobi WHOIS work showed a CA accepting a replacement-server email as a validation option — they stopped before issuing a rogue cert. Request counts are not infection counts.

*RetireSafe therefore treats every signal as sitting on an **evidence ladder**: stale reference → reclaimable endpoint → demonstrated impact. The product's value is moving a candidate up that ladder with proof, and refusing to assert a rung it hasn't shown.*

Three test beds, run on real data

Each test bed is a standalone script in `testbeds/` with a committed output file in `testbeds/results/`. All lookups are **read-only**: nothing was registered, claimed, or written to any third party.

TB1 · Base rate of surviving reclaimable references in the live internet

Data. Cisco Umbrella top-1M FQDNs (real DNS-resolver popularity). We sampled the top 8,000 and 8,000 uniformly-random names. **Method.** Walk each name's CNAME chain, match the tail against the *can-i-take-over-xyz* fingerprint catalogue (76 services), resolve the final target, and for AWS/S3 probe the bucket's existence via the S3 API (404 NoSuchBucket = reclaimable). Each name is placed on the evidence ladder; **raw hostnames are never published**.

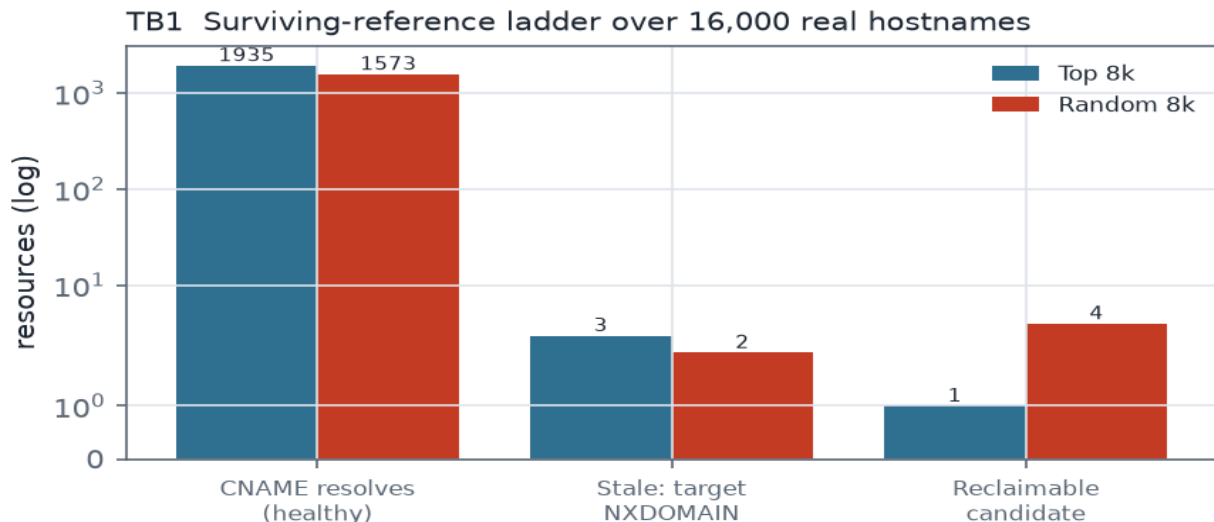


Fig 1 — Across 16,000 real hostnames the overwhelming majority resolve healthily; only a handful sit on the reclaimable rung. Counts are symlog-scaled.

Result. Of 16,000 live names, the pipeline flagged roughly **5 reclaimable candidates** (4 S3 NoSuchBucket, 1 Azure NXDOMAIN) plus 5 stale references — a base rate near **0.03%**. This is the single most important number for the product: the dangerous cases are *rare and buried*, which is precisely why manual review misses them and why an automated, evidence-first scanner earns its place. It also validates our fingerprint + live-probe pipeline end-to-end on real infrastructure.

TB2 · 'Is this endpoint really dead?' — a traffic-survival model

Data. NASA-HTTP July 1995 access log — **1,891,715** real requests, 7,133 distinct resources, 27.6-day span (Internet Traffic Archive). **Question.** The brief's core uncertainty: when an endpoint shows no hits for D days, how sure can we be that no consumer still depends on it? **Model.** Treat each resource's requests as a Poisson process with rate λ ; the chance of a silent window of length D is $e^{-(\lambda D)}$. Invert for the **quarantine**: $D \geq -\ln(\alpha)/\lambda$ with $\alpha = 1\%$.

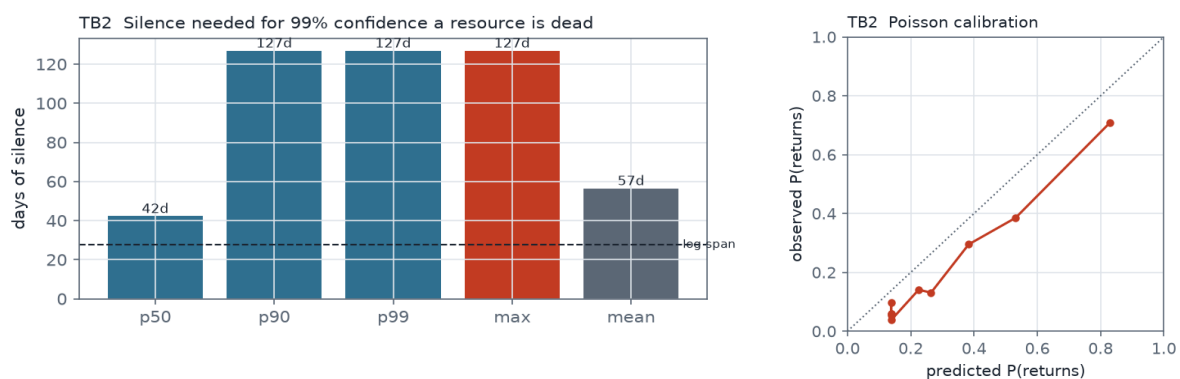


Fig 2 (left) — half of all resources need ≥ 42 days of silence, and the long tail needs ~ 127 days, to justify a 99%-confident 'dead' verdict. A single global idle-timeout is therefore unsafe. Fig 3 (right) — holding out the last week and estimating λ on the first 24 days, the Poisson model's predicted return-probability is consistently above the observed rate across every decile.

Result & honest finding. On the hold-out, 3,973 resources looked idle at the cutoff; the Poisson model expected 1161 of them to receive traffic again, and 783 actually did. The model **ranks** resources well (monotonic deciles) but is **mis-calibrated as an absolute probability** — traffic is bursty, not memoryless. The design takeaway is concrete: use the model to *prioritise and rank*, but gate deletion on an **empirically-calibrated, per-resource** quarantine with a safety margin, never on a parametric point estimate.

TB3 · The coordination gap, measured

Data. Same NASA log. **Idea.** The brief: a resource 'can appear unused to the team deleting it while remaining trusted elsewhere.' We make that measurable. Rank resources by how quiet they are in the final 3 days (what an activity dashboard would show), then reveal each one's **blast radius**: distinct client hosts, /24 networks, and *external* (non-nasa.gov) clients over full history.

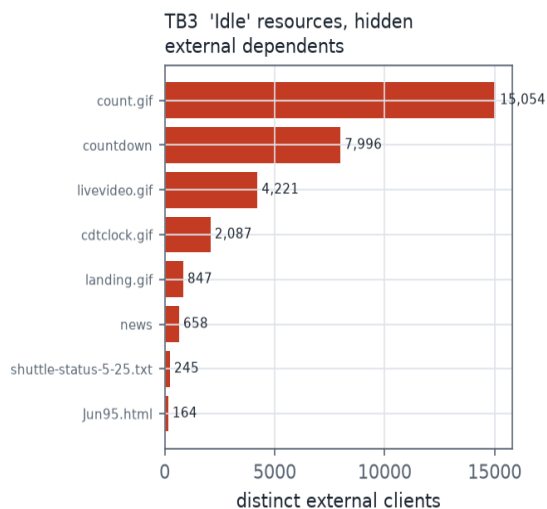


Fig 4 — the eight 'idle-looking' resources with the largest hidden external dependent populations.

Result. Of resources with real history that look idle in a 3-day window, **108 of 108 (100%)** still carried traffic from external clients. The worst case — /shuttle/countdown/count.gif — showed **0 hits** in the recency window yet **15,054** distinct external dependents across its history.

Deleting on the dashboard signal alone would silently break thousands of third parties the deleting team cannot see or notify. This is the coordination gap, quantified — and the direct justification for RetireSafe fusing DNS + code + **runtime traffic** before clearing a deletion.

S4

Proposed architecture

RetireSafe is a **retirement pre-flight**: given a proposed deletion, it gathers evidence from every place trust can hide, fuses it into one graph, and returns a per-condition verdict with a migration patch. Pipeline:

Stage	Component	Real inputs	Produces
Collect	Reference scanners	DNS/zone export, Git repo (regex+AST for URLs/buckets/SRI)Candidate (CNAMEs, the SOA, SPF/DNS TXT)	
Probe	Reclaimability prober	Fingerprint catalogue, S3/Azure existence + namespace rules, Bucket verification & protection	Each ref verifiable? bucket owner?
Observe	Traffic analyser	Access/CDN logs (the TB2/TB3 engine)	λ, quarantine, blast radius, external %
Fuse	Evidence graph	All of the above keyed by resource	Dependency graph + evidence per edge
Decide	Decision engine	Graph + \$5 risk score	Verdict: clear / block+reason / unknown
Act	Output layer	Verdict	Blocked plan, migration patch, evidence record

Outputs (from the brief's wish-list): a **blocked deletion with a precise reason**; a **proposed migration patch** (e.g. rewrite the CNAME, add ExpectedBucketOwner, drop the stale SPF include, set Terraform prevent_destroy); a **'retain the name' recommendation** when traffic persists; and an immutable **evidence record** stating inspection scope and what could *not* be checked.

Recognising existing protections is a first-class feature, not an afterthought: S3 account-regional namespaces (2026), bucket-owner-condition, Azure dangling-DNS detection + domain verification, GitHub Pages domain verification, and Terraform prevent_destroy each *downgrade* a finding's severity. RetireSafe must not cry wolf over a reference the provider already protects.

The mathematics

(a) Survival / quarantine. Per resource, $\lambda \blacksquare = n/T$ (MLE for a homogeneous Poisson process). Probability a truly-live resource shows zero hits over a silent window D is $P\blacksquare = e^{-(\lambda D)}$. Required quarantine for miss-tolerance α : $D^* = -\ln(\alpha)/\lambda \blacksquare$. TB2 shows D^* ranges 42→127 days at $\alpha=1\%$ — the engine emits D^* per resource.

(b) Uncertainty on rare resources. For a resource seen k times, a point rate is fragile. We bound the true request probability with the **Wilson** interval (better than normal approximation for small k) and, for counts, a Poisson/Gamma credible interval; the *upper* bound drives the conservative quarantine so infrequent-but-real clients are not discarded.

(c) Calibration, not blind trust. TB2's reliability diagram showed the raw Poisson is over-confident; the engine fits an **isotonic recalibration** on held-out history so the probability it reports ("12% chance a client still depends on this") is one an engineer can act on. We report Brier score / ECE, not just accuracy.

(d) Takeover risk score. Combine the five conditions multiplicatively — a chain is only as safe as its weakest cleared link:

$R = P(\text{released}) \cdot P(\text{reassignable}) \cdot P(\text{reference survives}) \cdot P(\text{consumer remains}) \cdot P(\text{controls fail to block}) \cdot \text{Impact}$

Each factor is an evidence-weighted probability with its own confidence; a factor that *cannot* be evaluated is reported as **unknown** and widens the score's interval rather than being silently assumed 0 or 1. **(e) Blast radius** (TB3): distinct clients C and /24 networks N over history, with external share $e = C_{\text{ext}}/C$, feed the Impact term and the human-readable 'who breaks' summary.

Status. Run on real data: (a) quarantine D^* , the hold-out calibration check in (c), and (e) blast radius. Proposed engine design, not yet run: the Wilson / Gamma bounds in (b), the isotonic recalibration fit in (c), and the risk score in (d).

Datasets & tools

Dataset / tool	Role	Status
Cisco Umbrella top-1M	Real popular-hostname sample (TB1)	Used, live DNS
NASA-HTTP 1995 (ITA)	1.89M real requests (TB2, TB3)	Used
can-i-take-over-xyz	76-service takeover fingerprint catalogue	Used
AWS S3 REST API	Live bucket-existence probe (read-only)	Used
dnspython	CNAME-chain + NS/A/AAAA resolution	Used
Terraform plan JSON	IaC deletion intent (prod scanner)	Designed
SPF/DNS TXT	Email-trust dependency (future scope)	Designed
NumPy/SciPy, Matplotlib, ReportLabModelling, figures, this PDF		Used

Hackathon scope & roadmap

Demo (build now). Legacy S3 buckets + a supplied repo + a DNS export + access logs + a proposed Terraform deletion. Show a controlled before/after: a deletion that is **blocked** with a precise reason and a migration patch, versus one **cleared** with its evidence record. **Next:** SPF/email-trust scanner (SubdoMailing class), Azure + GitHub Pages probers, CI/CD gate (a failing check on an unsafe terraform destroy), and continuous drift monitoring.

Honesty boundaries

- Our TB1 base rate is a sample estimate from 16k names, not a census; it establishes order-of-magnitude rarity, which is the point.
- TB2/TB3 use 1995 NASA traffic because it is a large, real, openly-licensed log; the *method* is dataset-agnostic and would run on a team's own CDN logs.
- A reclaimable candidate is a **candidate** — we never attempted a takeover. Demonstrating impact on a client's own resource, with authorisation, is future work.
- We recognise provider protections explicitly so we don't label protected references exploitable. The innovation is **safe-retirement coordination**, not rediscovering subdomain takeover.

Source register

[1] Microsoft Learn — Prevent dangling DNS & subdomain takeover. [2] Infoblox, 10 Mar 2025 — CDC incident. [3] Infoblox, 20 May 2025 — Hazy Hawk. [4] Guardio Labs, 26 Feb 2024 — SubdoMailing. [5] USENIX NSDI 2024, Friess et al. — *Cloudy with a Chance of Cyberattacks*. [6] watchTowr, 4 Feb 2025 — 8M requests, abandoned S3. [7] watchTowr, 11 Sep 2024 — .mobi WHOIS / RCE. [8] AWS, 24 Apr 2026 — S3 account regional namespaces. [9] AWS — bucket-owner-condition. [10] GitHub — Pages custom-domain verification. [11] HashiCorp — Terraform prevent_destroy. Additional tools: Cisco Umbrella top-1M; Internet Traffic Archive (NASA-HTTP); EdOverflow *can-i-take-over-xyz*.